DataCORE

Data Cleaning & R — Same Data, Different Tool

Afternoon: DataCORE

1. **Data Cleaning Workshop — Fixing Real Data**

2. **R Introduction — Same Data, Different Tool**

3. **Scientific Storytelling — Methods Language**

*Yesterday you described your data. Today we ask: **can you trust it yet?** First we clean it, then we meet a second tool — R.*

1:00 – 2:00 · Data Cleaning Workshop

🧹 Real data is messy — fix it systematically

- The clean datasets in textbooks **do not exist** in the wild.
- Real files have **missing cells**, **inconsistent labels**, and **impossible values**.
- Today we fix all three — in a **repeatable order** you can reuse on any dataset.

Today's pipeline:

1. Missing values → `fillna` / `dropna`
2. Inconsistent formatting → standardize
3. Outliers → identify, keep or flag

🛡️ **The golden rule: never overwrite your original data.** Save cleaning to a *new* file.

Load a messy dataset & look first

```
df = pd.read_csv("study_habits_messy.csv")    # original -- we will NOT overwrite this
print(df.head(6))
```

```
   student_id grade_level  gender study_method  hours_of_sleep  test_score
0           1         9th    Male         solo             7.5        82.0
1           2         9th  female        group             6.0        74.0
2           3         9th       M        tutor             8.0        88.0
3           4        10th  Female         Solo             5.5        68.0
4           5        10th    male        GROUP             7.0        79.0
5           6        10th       M         solo             NaN        71.0
```

Already spot trouble? `male` / `Male` / `M`, a blank cell, and a suspicious `999`.

`df.info()` — your first diagnostic

```
df.info()                          # column types + how many NON-missing values each has
```

```
<class 'pandas.DataFrame'>
RangeIndex: 22 entries, 0 to 21
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   student_id      22 non-null     int64
 1   grade_level     22 non-null     str
 2   gender          21 non-null     str
 3   study_method    22 non-null     str
 4   hours_of_sleep  20 non-null     float64
 5   test_score      20 non-null     float64
dtypes: float64(2), int64(1), str(3)
memory usage: 1.2 KB
```

A column with fewer non-null values than rows **has missing data**. Note `hours_of_sleep` and `test_score`.

Step 1 — Find the missing values

```
print(df.isna().sum())          # count of missing cells PER column
```

```
student_id        0
grade_level       0
gender            1
study_method      0
hours_of_sleep    2
test_score        2
dtype: int64
```

```
print("\nTotal rows:", len(df))
```

```
Total rows: 22
```

Now the decision: do we **drop** these rows, or **fill** them? The choice changes your results.

`df.dropna()` — remove rows with gaps

```
print("rows before:", len(df))
```

```
rows before: 22
```

```
print("rows after dropna():", len(df.dropna()))    # deletes ANY row with a missing cell
```

```
rows after dropna(): 18
```

Use `dropna()` when: missing rows are *few*, and dropping them won't bias your sample.

Risk: you throw away *every other value* in that row too — and may shrink small datasets fast.

`df.fillna()` — fill the gaps instead

```
median_sleep = df["hours_of_sleep"].median()      # robust center, ignores the 25 outlier
print("median hours of sleep:", median_sleep)
```

```
median hours of sleep: 7.0
```

```
filled = df["hours_of_sleep"].fillna(median_sleep) # fill -- shown, not yet committed
print(filled.head(6).tolist())
```

```
[7.5, 6.0, 8.0, 5.5, 7.0, 7.0]
```

Use `fillna()` when: you can't afford to lose rows; fill with the **median** (numbers) or **mode** (categories).

Risk: you're *inventing* values. Filling a key outcome variable can hide the truth.

⚖ `dropna` or `fillna`? Why the choice matters

Drop the row
A *few* rows missing, or the **outcome (DV)** is missing — you can't honestly invent a result.

Fill the value
Many rows affected and the gap is in a *secondary* column — impute the median / mode.

Always
Document it. Say how many rows you dropped or filled, and why — in your cleaning log.

💡 There is no universally "right" choice — there is only a **defensible, documented** one.

Step 2 — Inconsistent formatting

```
clean = df.copy()                       # work on a COPY -- protect the original
print("gender       :", clean["gender"].unique())
```

```
gender       : <StringArray>
['Male', 'female', 'M', 'Female', 'male', 'F', 'm', nan]
Length: 8, dtype: str
```

```
print("study_method :", clean["study_method"].unique())
```

```
study_method : <StringArray>
['solo', 'group', 'tutor', 'Solo', 'GROUP', 'Tutor', 'solo ', 'Group']
Length: 8, dtype: str
```

⚠ The computer reads `'Male'`, `'male'`, and `'M'` as **three different groups**. We must merge them.

🪄 Standardize string columns

```
gender_map = {"male": "Male", "m": "Male", "female": "Female", "f": "Female"}
clean["gender"] = clean["gender"].str.strip().str.lower().map(gender_map)    # 3 steps
clean["study_method"] = clean["study_method"].str.strip().str.lower()        # trim + lower

print(clean["gender"].value_counts())
```

```
gender
Male      12
Female     9
Name: count, dtype: int64
```

```
print()
```

```
print(clean["study_method"].value_counts())
```

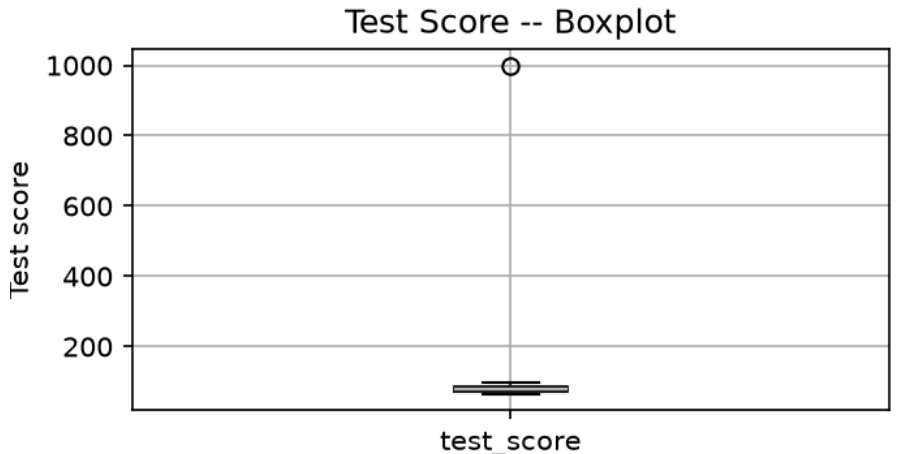
```
study_method
solo     8
group    7
tutor    7
Name: count, dtype: int64
```

- `.str.strip()` removes stray spaces, `.str.lower()` unifies case, `.map()` collapses to one label.

Step 3 — Spot outliers with a boxplot

Test score | Hours of sleep

```
clean.boxplot(column="test_score")
plt.title("Test Score -- Boxplot"); plt.ylabel("Test score")
plt.show()
```



- Dots far past the whiskers are **suspects**: a `999` test score and a `25`-hour night are *impossible*, not just extreme.

🏳 Keep or flag? Make & record the decision

```
# These values are physically impossible -> flag as missing (a documented decision)
clean.loc[clean["test_score"]  > 100, "test_score"]      = np.nan   # max score is 100
clean.loc[clean["hours_of_sleep"] > 12, "hours_of_sleep"] = np.nan # nobody sleeps 25 hrs
print("flagged as missing:", df.shape[0] - clean[["test_score","hours_of_sleep"]].notna().all(axis=1).sum(), "rows affected")
```

```
flagged as missing: 5 rows affected
```

📘 **Not every outlier is an error.** A genuine extreme value you *keep* (and flag); an impossible value you *fix*. Either way — **write it in the log.**

Your data cleaning log

A running record — one row per decision — so your cleaning is **reproducible and defensible**.

Issue found	Column	Decision	Why
Missing values	hours_of_sleep	Filled with median	Secondary variable; few rows
Missing values	test_score	Dropped those rows	Can't invent the outcome
Mixed labels	gender, study_method	Standardized to one label each	Same category, 3 spellings
Impossible value (999)	test_score	Flagged as missing	Score cannot exceed 100
Impossible value (25)	hours_of_sleep	Flagged as missing	Not physically possible

Keep this in your shared notebook — it becomes the backbone of your **Methods** section.

Finish & save as a NEW file

```
clean["hours_of_sleep"] = clean["hours_of_sleep"].fillna(clean["hours_of_sleep"].median())
clean = clean.dropna(subset=["test_score"])      # drop rows missing the outcome

clean.to_csv("study_habits_clean.csv", index=False)    # NEW file -- original untouched
print("original :", df.shape, "-> study_habits_messy.csv")
```

```
original : (22, 6) -> study_habits_messy.csv
```

```
print("cleaned  :", clean.shape, "-> study_habits_clean.csv")
```

```
cleaned  : (19, 6) -> study_habits_clean.csv
```

Never `to_csv` over your raw file. If cleaning goes wrong, you must be able to start over.

AI assist — generate, then *test* the code

Prompt the AI:

> *"My column has these unique values:* `['Male','male','M','Female','female','F']`*. Write Python code to standardize them into two categories."*

Paste in **your** real unique values from `df["col"].unique()`.

Then test it — does it handle edge cases?

- A trailing space: `'male '`?
- An unexpected value: `'Nonbinary'`, `'?'`
- A **missing** cell (`NaN`)?
- A new spelling next week?

Run it on your data and check the `value_counts()`.

⚠ AI writes plausible code fast — **you** are responsible for proving it's correct.

Team Workshop

🏁 Apply all three techniques — ~25 min

On your team's real dataset:

1. `df.isna().sum()` → decide **drop vs fill** for each column.
2. Standardize every **string** column to one label per category.
3. Boxplot your numeric columns → **keep or flag** outliers.
4. Save as **`yourdata_clean.csv`** — a *new* file.

Output before you leave:

- ✓ A clean CSV (original untouched)
- ✓ A completed **cleaning log**
- ✓ Row count: before → after

25:00

2:00 – 2:45 · R Introduction

Meet R — not better, just *different*

- Open **RStudio Cloud** — free, browser-based, nothing to install.
- We'll load the **same dataset** and run the **same summaries** you ran in Python.
- **R is not better or worse than Python.** It's a different tool, favored in much **social science & health** research.

Why learn both? Researchers pick tools by **field norms** and **the task** — not loyalty. Being bilingual makes you flexible.

Same data · same questions · a second set of words to ask them in.

`read.csv`, `head`, `summary` — the R equivalents

Python (pandas) | R (base)

```
df = pd.read_csv("study_habits.csv")
print(df.head())
```

```
   student_id grade_level study_method  hours_of_sleep  test_score
0           1         9th         solo             7.5        82.0
1           2         9th        group             6.0        74.0
2           3         9th        tutor             8.0        88.0
3           4        10th         solo             5.5        68.0
4           5        10th        group             7.0        79.0
```

Same idea, different spelling: `pd.read_csv()` → `read.csv()`, `df.head()` → `head(df)`.

`summary()` — R's `df.describe()`

R | Python

```r
summary(df)                          # pandas: df.describe()
```

```
  student_id    grade_level        study_method       hours_of_sleep
Min.   : 1.00   Length:20          Length:20          Min.   :5.000
1st Qu.: 5.75   Class :character   Class :character   1st Qu.:6.000
Median :10.50   Mode  :character   Mode  :character   Median :7.000
Mean   :10.50                                         Mean   :7.053
3rd Qu.:15.25                                         3rd Qu.:8.000
Max.   :20.00                                         Max.   :9.000
                                                      NA's   :1

  test_score
Min.   :65.00
1st Qu.:73.00
Median :80.00
Mean   :80.42
3rd Qu.:88.50
Max.   :96.00
NA's   :1
```

Notice R's `summary()` describes **every** column at once — including the text ones.

RcS — install `dplyr`, then `filter()` & `mutate()`

```
install.packages("dplyr")      # run ONCE per environment (RStudio Cloud)
```

```
library(dplyr)

df |>                                          # the pipe: "and then..."
  filter(grade_level == "11th") |>             # keep rows  (pandas: df[df.col == ...])
  head(4)
```

```
  student_id grade_level study_method hours_of_sleep test_score
1          8        11th        group            6.5         77
2          9        11th         solo            7.0         80
3         10        11th        tutor            9.0         95
4         11        11th        group            5.0         65
```

`filter()` keeps **rows** that meet a condition — like a query on your data.

➕ `mutate()` — add a new column

```
df |>
  mutate(enough_sleep = ifelse(hours_of_sleep >= 7, "yes", "no")) |>    # new column
  head(5)
```

```
  student_id grade_level study_method hours_of_sleep test_score enough_sleep
1          1         9th         solo             7.5         82          yes
2          2         9th        group             6.0         74           no
3          3         9th        tutor             8.0         88          yes
4          4        10th         solo             5.5         68           no
5          5        10th        group             7.0         79          yes
```

🪄 `mutate()` **creates or changes columns** — the R cousin of `df["new"] = ...` in pandas.

⚖️ Two tools, one job — what did you notice?

What feels easier in Python?

- One object `df` with dot-methods
- Huge ecosystem for ML & web data
- Very common in industry & CS

What feels cleaner in R?

- `summary()` describes everything at once
- The pipe reads like a sentence
- Built *by* statisticians, for stats

No answer is wrong. Note one thing you liked in each — researchers choose tools by field norms and task requirements.

2:45 – 3:00 · Scientific Storytelling

Two sentences for your Methods slide

Write **two sentences** — the start of your written Methods section:

1. **Describe your dataset** — name, source, timeframe, **N**.
2. **Describe how you cleaned it** — what you dropped, filled, or fixed.

Write in the **past tense** — the work is done: *"We used...", "We excluded..."*

1. The dataset
*"We used the **Study Habits Survey** (school records, 2024–25), covering **N = 19** students after cleaning."*

2. The cleaning
"We excluded rows where the test score was missing, standardized gender labels, and flagged impossible values as missing."

Write yours now — quick round

Fill in the template on your Mission Sheet, then read **both sentences** aloud to a partner.

> *"We used **[dataset]** from **[source]**, covering **[timeframe]**, with **N = [count]**. We cleaned it by **[what you did]**."*

Checklist:

- Past tense? (*"We used..."*)
- Name **and** source of data?
- Timeframe **and** N?
- One concrete cleaning step?

15:00